

CO 3 - ASSESSMENT TOOL -2

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA1003
Subject Name : Software Engineering

Hotel Reservation and Room Management Platform with Dynamic Pricing

Code Review & Repository Evaluation

Hotel Reservation and Room Management Platform with Dynamic Pricing

1. Problem Overview

The **Hotel Reservation and Room Management Platform with Dynamic Pricing** is a full-stack software application designed to automate hotel room reservation, room management, customer management, and dynamic pricing.

The platform allows customers to register, log in, search for available rooms, view room details, check prices, and make reservations. Hotel administrators can manage rooms, reservations, customers, room availability, pricing rules, and booking information.

The main feature of the system is the **Dynamic Pricing Module**. The room price is automatically adjusted according to factors such as room availability, booking demand, occupancy level, season, day of the week, and special events.

Objectives :

- Provide an online hotel room reservation system.

- Allow customers to search and reserve rooms.

- Maintain accurate room availability.

- Prevent double booking.

- Allow administrators to manage rooms and reservations.

- Automatically calculate room prices.

- Improve hotel revenue using demand-based pricing.

- Provide a centralized platform for hotel management.

Maintain customer and reservation information securely.

Support scalable deployment using Docker.

2. Repository Organization



Repository Structure Evaluation

Folder/File	Purpose
frontend/	Contains the user interface
public/	Contains static frontend files
components/	Contains reusable UI components
pages/	Contains application pages
services/	Handles frontend API communication
hooks/	Contains reusable React hooks
context/	Manages application state
backend/	Contains server-side application logic
controllers/	Handles incoming API requests
models/	Defines database models
routes/	Defines API endpoints
services/	Contains business logic including dynamic pricing
middleware/	Handles authentication and request processing
validators/	Validates user and reservation data

Folder/File	Purpose
database/	Contains database initialization scripts
docs/	Stores architecture and workflow documentation
docker-compose.yml	Defines multi-container deployment
.gitignore	Prevents unnecessary files from being committed
README.md	Contains project setup and usage instructions

This organization improves maintainability because frontend, backend, database, and documentation components are separated. It also supports collaboration because developers can work on individual modules without unnecessarily modifying unrelated files.

The assessment specifically requires evaluation of repository structure, folder organization, naming conventions, maintainability, and collaboration.

3. Code Quality Review

The code is organized using a modular architecture. Different responsibilities are separated into controllers, models, routes, services, and middleware.

Code Readability

The code should use:

- Meaningful variable names.

- Meaningful function names.

- Consistent indentation.

- Proper spacing.

- Simple and understandable logic.

- Comments for complex algorithms.

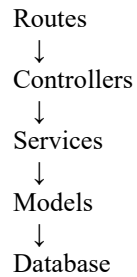
Example:

```
function calculateDynamicPrice(basePrice, occupancy, demandFactor) {
  return basePrice * occupancy * demandFactor;
}
```

The function name clearly describes its purpose, making the code easier to understand.

Modularity

The application follows modular design by separating:



For example, the dynamic pricing calculation is maintained inside the pricing service rather than being mixed directly with reservation code.

Coding Standards

The project should follow:

- Consistent naming conventions.
- Proper indentation.
- Modular functions.
- Avoidance of duplicate code.
- Proper error handling.
- Input validation.
- Secure environment-variable management.

Strengths :

- Modular project structure.
- Separation of frontend and backend.
- Reusable components.
- Dedicated dynamic pricing service.
- Organized API routes.
- Centralized database access.
- Docker support.
- Git-based version control.

Areas for Improvement :

Add automated unit testing.

Improve error handling.

Add API documentation.

Introduce code linting.

Add input validation to all APIs.

Improve logging.

Add automated security checks.

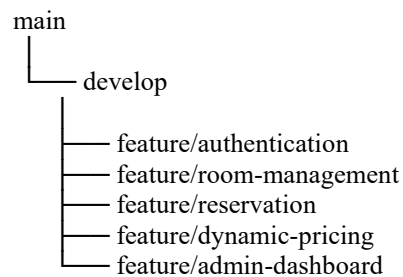
Increase test coverage.

The assessment evaluates readability, modularity, coding standards, documentation, and maintainability as major code-quality criteria.

4. Version Control Evaluation

Git is used to maintain different versions of the hotel reservation platform.

Branching Strategy



Commit Message Evaluation

Good commit:

```
git commit -m "Implement dynamic room pricing"
```

Poor commit:

```
git commit -m "changes"
```

Meaningful commit messages make it easier to understand what changed and why.

Version Control Benefits

Git supports:

Collaboration.

Version tracking.

Branch-based development.

Code review.

Rollback.

Conflict resolution.

Repository maintenance.

The assignment specifically requires analysis of commit history, branching strategy, commit messages, and repository management practices.

5. Code Testing and Validation

Testing is performed to verify that all major functions of the hotel platform operate correctly.

Test Cases

Test Case	Input/Action	Expected Result
TC01	Register new user	User account created
TC02	Login with valid credentials	User successfully logged in
TC03	Login with invalid password	Error message displayed
TC04	Search available rooms	Available rooms displayed
TC05	Select room	Room details displayed
TC06	Make reservation	Reservation successfully created
TC07	Book unavailable room	Booking rejected
TC08	Cancel reservation	Reservation cancelled
TC09	Increase room demand	Room price increases
TC10	Low room demand	Price adjusts downward
TC11	Admin adds room	New room displayed
TC12	Admin changes room status	Room availability updated
TC13	Invalid reservation data	Validation error displayed
TC14	Database connection	Data successfully retrieved
TC15	Docker deployment	All services run successfully

Dynamic Pricing Validation

Example:

Base Room Price = ₹3000

Low Demand



Price = ₹2700

Normal Demand



Price = ₹3000

High Demand



Price = ₹3600

The dynamic pricing module must ensure that price changes are calculated correctly according to the configured pricing rules.

6. Repository Documentation

The project should contain a README.md file with complete setup and usage information.

README Contents

Project Title
Project Description
Features
Technology Stack
System Requirements
Installation
Database Configuration
Environment Variables
Running the Frontend
Running the Backend
Running with Docker
API Documentation
Testing
Git Workflow
Project Structure
Future Enhancements

Installation

The documentation should explain how to install project dependencies.

Frontend:

```
cd frontend
npm install
npm run dev
```

Backend:

```
cd backend
```

npm install
npm start

Docker:

docker compose up --build

Documentation Improvements

The documentation can be improved by adding:

Screenshots.

API documentation.

Database schema.

Architecture diagram.

Troubleshooting section.

Docker deployment instructions.

Example API requests.

Testing instructions.

Contribution guidelines.

The assignment requires evaluation of the README, installation instructions, usage guide, dependencies, and documentation completeness.

7. Code Review Findings and Recommendations

Recommendations :

Code Quality :

Use ESLint and Prettier.

Follow consistent naming conventions.

Avoid duplicate code.

Implement centralized error handling.

Add stronger input validation.

Repository Management :

- Use protected main branch.
- Require pull requests for major changes.
- Use feature branches.
- Maintain meaningful commit messages.
- Tag stable releases.

Testing :

- Add unit tests.
- Add integration tests.
- Test reservation conflicts.
- Test dynamic pricing under different demand levels.
- Add API testing.
- Increase automated test coverage.

Documentation :

- Maintain an updated README.
- Add API documentation.
- Include architecture diagrams.
- Document database tables.
- Provide Docker setup instructions.

Future Enhancements :

- Machine-learning-based demand prediction.
- AI-based room recommendations.
- Online payment integration.
- Kubernetes orchestration.
- Real-time pricing updates.

Conclusion

The **Hotel Reservation and Room Management Platform with Dynamic Pricing** provides a structured solution for hotel reservation and room management. The modular repository organization improves code maintainability, while Git provides effective version control and collaboration. Docker provides a consistent and portable deployment environment.

The code review identifies the importance of readable, modular, documented, and testable code. Further improvements such as automated testing, stronger documentation, machine-learning-based demand prediction, advanced analytics, and cloud deployment can make the platform more scalable and production-ready.

Overall, the project satisfies the major areas evaluated in the Code Review & Repository Evaluation: **problem analysis, repository organization, code quality, version control, testing, documentation, recommendations, and technical presentation**. The assessment rubric gives specific weight to these areas, including 25% for code quality, 20% for version control, 15% for testing, and 15% for documentation and repository maintenance.